NICHOLAS

https://www.canva.com/design/DAHZeG_HELQ/ooWlOpAyYi8gnAWG0h9Lug/edit?ui=e30

[CANVA Slide 1]
Have you ever used an NTU portal and thought to yourself, “man, this entire user experience is terrible, why did they implement it like this?” That’s exactly what me and my team thought when we accessed the URECA portal for the first time earlier this year, and that’s how we ended up developing URECAMatch.

[CANVA Slide 2]
But first, what exactly is URECA? Quoting directly from the NTU website, it stands for Undergraduate Research Experience on Campus. It is a year-long research project done by undergraduates under the supervision of a supervisor, most often a professor or faculty staff. Projects can either be put forward by a Supervisor, or self-proposed by a student.

That’s all well and good, but at this point, you might be wondering, what gap are we trying to solve with our web portal? Now, this all becomes clear in a moment.

[CANVA Slide 3]
This is what the current portal for URECA applications looks like. First, the standard NTU login screen we’ve all come to know and love.

[CANVA Slide 4]
After login, you’re greeted with this page. On this page, you can see every single project available to apply for.

And that’s it. That’s the entire current application “portal” and the entire process. It’s basically just a glorified PDF list of projects. To apply, you have to email the professors yourself.

You can see immediately there’re a few problems. First, the application process is not well-structured, and as a student, emailing multiple professors at once might be considered rude. But if you email only one professor, what happens if you get ghosted, and in the meantime lose out on opportunities to apply for your 2nd choice projects?

As a Supervisor, managing multiple applicants (especially if you posted multiple projects) in your email inbox with separate CVs, transcripts will be a headache to deal with.

Last but not least, as an Administrator, you have little control over the process, and limited powers to step in and help or arbitrate as communication takes place directly between students and professors.

[CANVA Slide 5]
And so, presenting our app, the URECAMatch Webportal!

[CANVA Slide 6]
We have 3 main target users: Admins, Supervisors, and of course, Students.

[CANVA Slide 7]
We aim to solve all of those issues we covered earlier, and more. For admins, a unified interface with more powers to oversee the application process and arbitrate if necessary. For Supervisors, a centralized platform to manage applicant data, and for students, a more structured application process with clear deadlines and results announcements.

[CANVA Slide 8]
But before our product demo, some necessary housekeeping information. This is how we structure our entire application process. Every year, the admin creates one Application Cycle. Each Cycle has a pre-application window, where Students can view posted projects and edit their profiles but can't apply. Next we have rounds. Each cycle has 3 rounds, and each round consists of 3 windows. The application window, where students will apply for projects, the Review Window, which is the deadline for Supervisors to consider their applicants and submit their selections, and the Approval Window, where the system will run the matching algorithm and give each Supervisor a proposed Student. If the Supervisor has no problems with the proposed Student, the offer is then given to the Student to accept or decline, and the Round then repeats.

[CANVA Slide 9]
And now, with all the boring stuff out of the way, time for our actual product demo. Take it away, Robin.

ROBIN

[Open the link > Sign In page > "Create account"]

Now I will be Jason, a student who wants to apply for a URECA project. Jason opens the link and lands on the URECAMatch Sign In page. He is a first timer, so he creates an account first. He enters his full name, Jason Tan, his NTU email and a password.

[Verification code page > enter a wrong code first > then the correct code]

A verification code is sent to his email. If he types a wrong code, it is rejected. Only the valid code works. Once the account is activated, Jason can sign in.

[Sign in > Profile page > upload transcript]

After signing in, Jason must complete his profile before using the system. He uploads his NTU transcript, and only a valid NTU transcript is accepted. URECAMatch then fills in his details automatically. Jason is a Computer Science student, these are the modules he has taken, and his GPA is 4.67, all correctly reflected here.

[Profile > upload CV > research interests: deep learning, machine learning, robotics > Save]

Jason can also upload his CV to show his experience. He has a very high GPA, but not much relevant experience. We will see how URECAMatch picks that up later. He also picks his research interests: deep learning, machine learning and robotics. Once he saves, he can use the platform.

[Sidebar > Browse Projects]

Under Browse Projects, URECAMatch recommends the best fit projects using three things: the interests he picked, his readiness (does he meet the prerequisites), and his chance (fewer applicants means a higher chance). Jason picked deep learning and machine learning, he has the prerequisite modules, and this project has no applicants yet, so it is recommended to him. He can still view other projects if he wants.

[Click the Deep Learning project]

Jason opens the project. He can see how many slots are open, how many students have applied, the weekly time commitment, what he will do, and some related pictures and a video.

[Apply > fill statement > select documents > apply button]

Jason is interested, so he clicks Apply. He has to write an interest statement and choose supporting documents. Jason is a bit lazy. He thinks his high GPA will secure the project, so he writes a quick statement and does not care much about his CV. Before submitting, he uses the AI check. It says his statement is weak and his CV is weak. For now Jason ignores that and clicks Apply. We will see what happens.

[Sidebar > My Applications]

All his applications are listed here. If a student applies to several projects, they can rank them by preference.

[Self-propose project > pick supervisor > fill in details > Submit]

Students can also propose their own project. Jason likes another deep learning topic, but cannot find it in the listings. So he picks a supervisor, fills in the details, and hopes it gets accepted.

[Sign in as Wei Ling > Profile]

Now the second student, Wei Ling. She is also a Computer Science student. Her GPA is lower than Jason's at 3.99, but she has a very strong portfolio, with project and hackathon experience. She has the same interests as Jason.

[Browse Projects > Deep Learning project]

She goes to the same project. We can see one slot is open and one student, Jason, has already applied.

[Apply > fill statement > Apply button]

Wei Ling has a lot of experience, so she writes a detailed statement. When she uses the AI check, it says her statement and her CV are strong. She applies.

[Sidebar > My Applications > rank Deep Learning first]

She has applied to several projects, so she ranks her preferences and puts Deep Learning first.

Now I will pass over to Ian. Who gets the project: Jason with the high GPA, or Wei Ling with the strong CV?

IAN

[Advance time to the review window using the debug clock]

Step 1: Create and edit a listing

[My projects > New project]
First, supervisors can post a new project. It's a four-step guided form.

[Step 1: paste title and description > Continue]
I start with the title and description, which is what students see first.

[Step 2: pick topic, pick prerequisite, type location, + to 2 slots > Continue]
Next, the topics and prerequisites come from standard lists, so students can filter by them and see which modules they're missing. I'll take two students on this project.

[Step 3: hours, mode, video link > Continue]
Then the workload, the mode, and a video about the project.

[Step 4: add cover and 3 photos > Save as draft]
Finally the cover and photos. I can save a draft at any point.
[Drafts tab > Publish]
Students only see a project once it's published, so I'll publish it now.
[Published tab > Edit on the new listing > change description > Save changes]
I can still edit it until the cycle closes. The one limit is that slots can't drop below what's already filled or held.

Step 2: Applications and self-proposed projects

[My projects > Applicants on Graph Learning > click Student #1]
Now let's see who applied. This is the Graph Learning project our two students just applied to. Clicking a student shows the whole application in one place: CGPA, modules, whether they meet the prerequisites, their interests, statements and documents.

[Esc > Proposals in the sidebar > Student #1's card > View profile > close]
Students can also propose their own project. Here's the one Student #1 sent me.

[Decline > paste the reason > Decline]
The scope is too broad for one semester, so I'll decline it. A reason is required, because the student reads it and can improve the proposal for the next round.

[My projects > Applicants on Graph Learning > + on Student #2 > + on Student #1]
Back on this project, there's only one slot, so my order is decided. The stronger applicant, Student #2, go first.

Step 3: The ranking page

[My projects > Applicants on Deep Learning for Early Detection of Diabetic Retinopathy]
To show the ranking tools properly, I'll open a project with more applicants. There are eight, and only one meets the prerequisites. Four aren't ranked yet, and the page warns me they can't be matched.

Step 4: Filtering

[Filters > School > CCDS > Meets prerequisites tab]
I can filter by CGPA, school and year, or search by name. Here I'm looking at CCDS students who meet the prerequisites. The filters are kept in the address, so a refresh won't lose them.
[Clear all > All tab]

Step 5: Gemini

[Weights > filtering]
The supervisor is able to weigh what he deems as important to him, if he does not value CGPA as much he is able to tailor the percentage accordingly making the weightage for past experience higher than the rest

[AI match > wait about 5 seconds > hover a score]
With many applicants, reading every statement takes time, so I can ask Gemini for help. Each student gets a fit score out of 100 with a short reason in accordance to the weightage he has applied. It reads the statement as well as the grades.

[Results of matching]
As we can see the student that got the higher experience but lower CGPA has been ranked as 1st while the student with a higher CGPA but lower experience is ranked lower, This is only a guide. It never changes the final ranking or the matching. The decision stays with the supervisor.

Step 6: Manual sort

[View students profile]
I can view more info on the particular student clicking on their profile, i can have a indepth of their profile and choose to download the documents they have attached.

[+ on one student > up arrow on another > remove a third]
I can build the ranking by hand: add a student, move someone up, or remove someone. It saves automatically, so there's no save button.

Step 7: Automatic sort

[Check the sort shows AI score > type 2 in Top > Rank > Replace ranking]
Or I can do it in one step. This project has three places and one is filled, so I want two more. I type two, and it ranks the first two applicants as currently filtered and sorted. I can still adjust it after.

Handover

That's the supervisor's side. I'll hand it over to Nicholas, who will show what the admin can do

NICHOLAS

[Sign in as Admin: Jonathan Teo] [email and password input needed]

Next, let's login to an Administrator Account.

[Scroll down dashboard]

This is the dashboard the Admin sees. Immediately, you can see it's a huge improvement over the current portal, because Admin can see all sorts of information about the entire application process, and this sort of data might even be helpful for improving future iterations of the programme.

[Sidebar > Application Cycle > Click Round 1,2,3 in order]

So first, we see how an application cycle is set up. You can see over here that there's an Application Cycle currently in progress, and we're in Round 2, along with each window's start and end dates, and summary of each round.

[Open Cycle Settings (top right)]

This is where we configure the dates and settings for the Application Cycle. All of these greyed out portions are rounds that have elapsed or are in-progress, and cannot be changed.

[Still on "Cycle Settings" > Change Round 3 Start Date to 11/11/2026 (error)]

When changing or modifying the application startdates/deadlines, there are safeguards in place as well. For example, if you try to start Round 3 before Round 2 even ends. You get an error and the reason why.

[Extend Round 3 Approval to 5 days > Observe Error Message and Recommendation]

Now, let's try to extend the approval window of Round 3 to 5 days. The system warns you that the overall application date is actually too close to the end of this window, and students might not have time to respond. So let's take their recommendation here and extend the end of the application cycle accordingly.

[Click on "Extend to 2 Dec" > Close Cycle Settings]

So let's take the System's recommendation here and extend the end of the application cycle accordingly. And there, it's changed.

[Sidebar > Assignments]
(optional: show alternate path from Application Cycles > Round 1 > "View Matches" > "Assignments")
Next, if something goes wrong in the application process, you want the Admin to have the power to override the system and manually assign or unassign students from Projects. For example, if a student accidentally declines their offer, or if a student withdraws from their degree programme

[Top Filter Bar > "No project" > Manual Assign first student "Aisha Rahman" to any Project]
[Justification text input needed]

Let’s see an example of manually assigning a project. We filter students that are not assigned to any project, and at top of the list, we have Aisha Rahman, wLet’s give her a project. When Admins manually override the System like this, their actions are logged and they have to provide a justification. And there, it’s done!

[Top Filter Bar > “Accepted” > Manual Unassign first student “Divya Nair”] [Justification Text input needed]

Now, let’s do the opposite and unassign a student from a project. The chosen lucky student is Divya Nair. And there, unassigned.

[Sidebar > Audit Log] (optional: show Project Status to see that manual overrides worked)

As you can see important actions taken by Users are here are logged into URECAMatch. In particular, when I as the Admin overrode the System using manual assignment just now, it’s recorded here for accountability.

Let’s now fast forward time to the approval window, after the matching algorithm runs, and Supervisors will be given a prospective student to consider.

[Advance time to approval window using the debug clock]

[Sidebar > Either Dashboard or Application Cycles to see that we’re now in approval window]

Dashboard now shows we’re in the Approval Window of Round 2. I’ll pass over to Robin for the remaining time.

[Pass over to Robin]

Robin, have at it.

ROBIN

Thank you, Nicholas. Now the part we are all excited about: the results.

[Sign in as Wei Ling > Dashboard or My Applications > View Offer]

Let's start with Wei Ling. She got an offer. Her GPA is lower than Jason's, but experience matters, and her strong statement made her stand out. She has the project.

[My Application > Accept]

Wei Ling accepts the offer. Her cycle is now closed, so she cannot apply for any more projects.

[Sign in as Jason > My Applications]

Now let's look at Jason. He has a high GPA, but Deep Learning has only one slot, and Wei Ling took it. So his application is rejected.

[My Applications > self-proposed project > view decision note]

His self-proposed project is rejected too. The supervisor's feedback says the statement was not good enough.

[Advance time to Round 3]

Now we move on to the Round 3 window.

[Browse Projects > Deep Learning project]

Deep Learning is already full because of Wei Ling, so Jason cannot apply again.

[My Applications > self-proposed project > edit statement > Resubmit]

Instead, Jason improves his project statement using the professor's feedback. This time he spends time and effort to write it properly, then submits it again.

[Sign out > sign in as the supervisor > self-proposed projects > Accept]

The professor reads it, is happy with it, and accepts the proposal.

[Sign in as Jason > My Applications]

Jason also gets a project, his own proposal, and his cycle ends as well.

[Open Jason's email inbox > show the notifications] 

Here are the email notifications. URECAMatch tells Jason about every step by email, so he does not need to keep checking the platform.

That brings us to the end of our presentation. Thank you.